



File Management

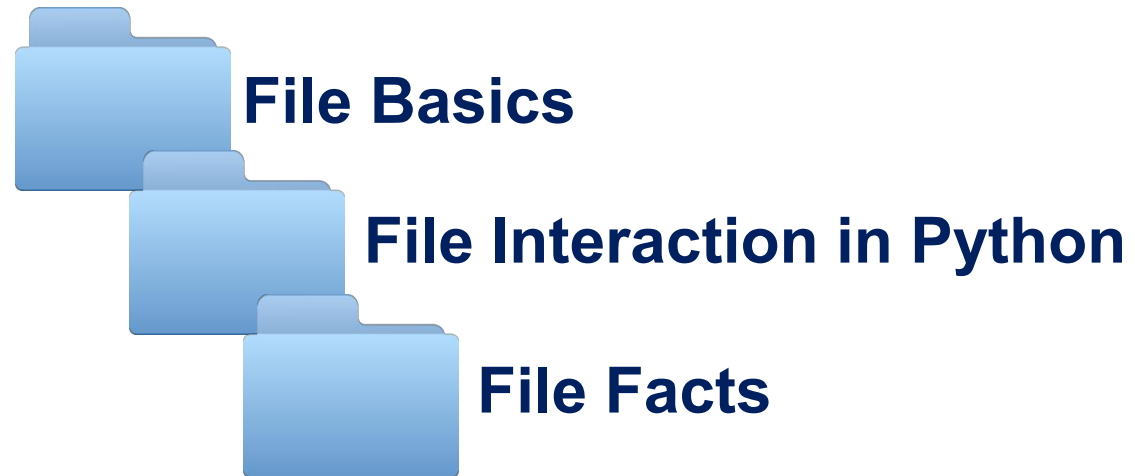
Lesson Objectives



At the end of this lesson, you should be able to:

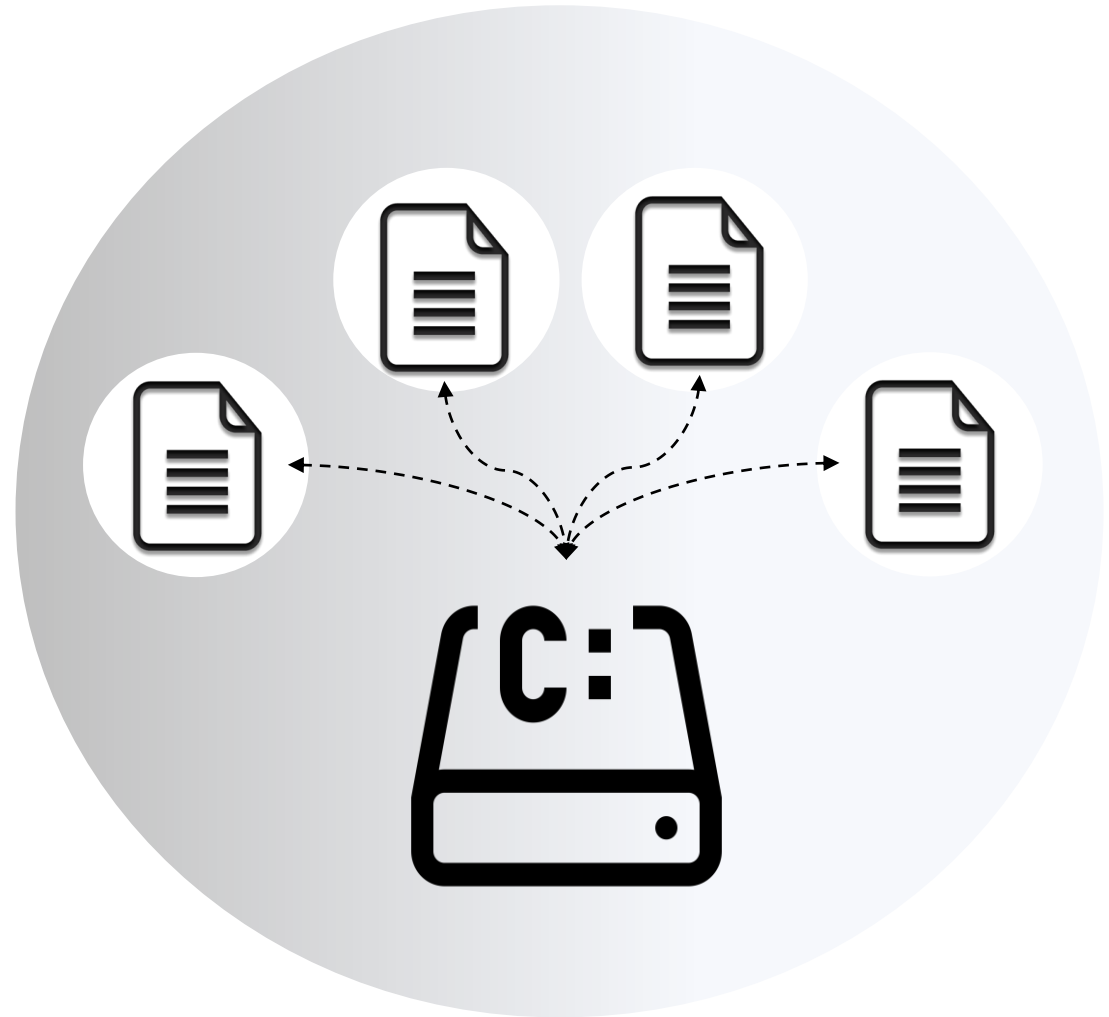
- Explain the concept of files
- Use the Python programming language to interact with files

Topic Outline



File Basics

- A **collection of data** that is stored on **secondary storage**, e.g. a disk
- Accessing a file means establishing a **connection** between the **file** and the **program** and moving data between the two.



Two Types of Files

Files come in two general types:

1. Text files

- Organized as ASCII or Unicode characters
- Generally human readable
- Main focus of this course



2. Binary files

- All the information is based on specific encodings
- Not human readable and contains non-readable information



Binary vs. Plain Text



Plain Text

- + Human readable, useful for certain file types
- Inefficient storage (each character requires ? bytes) :
 - ASCII: 8 bits $\rightarrow$ 1byte
 - Unicode: 32 bits $\rightarrow$ 4 bytes

Binary

- + More efficient storage, custom format
- Not human readable



Example



Assume there are 500 students in a class. If all the ages of the students were stored in both types of files, how many bytes would each entry be?

ASCII

'20'
'18'
'21'
'19'

ASCII = 2 bytes (two characters)
 $2 \times 500 = 1000 \text{ bytes}$

Binary

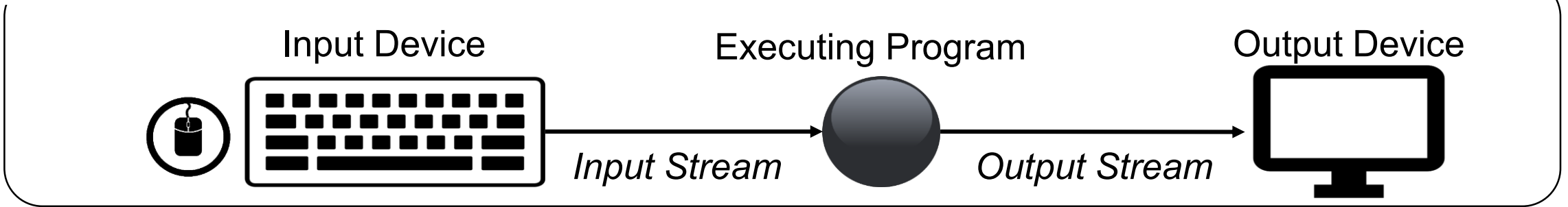
20
18
21
19

Binary = 1 byte
1 byte is 0-255 (enough for age)
 $1 \times 500 = 500 \text{ bytes}$

File Interaction in Python

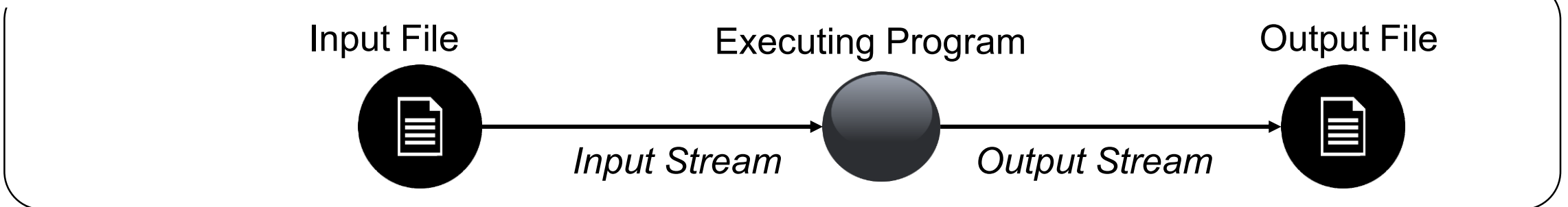
File Object or Stream

a. Standard Input and Output

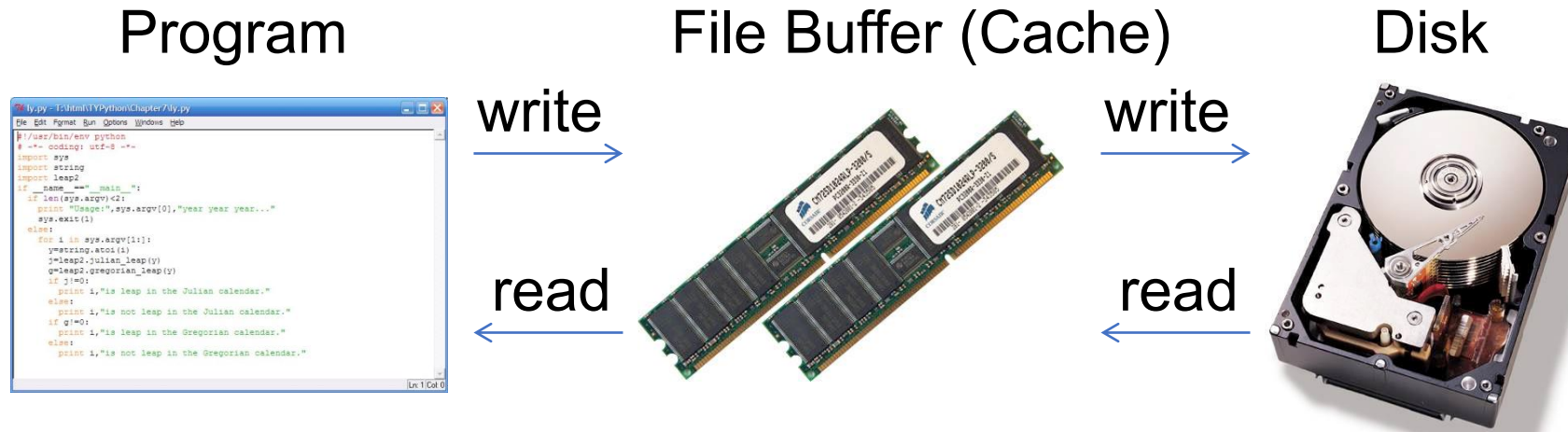


When **opening** a file, you create a **file object** or **file stream** that is a connection between the **file** and the **program**.

b. File Input and Output



- Reading from (writing into) a **disk** is **very slow**.
- A computer tries to read a lot of data from a file first
 - if you need the data, it will be **“buffered”** in the file object.
- The file object contains a copy of information from the file called a **cache** (pronounced “cash”).



Where Does the “Buffer” Reside?

- The **file buffer** contains the information from the file and provides the information to the program.
- Located in the **file object**



Creating a File Object

```
myFile = open("myFile.txt", "r")
```

- `myFile` is the file object.
- It contains the *buffer* of information.
- The first quoted string is the **file name** on disk, the second is the **mode** to open it (here, "r" means to read).

When opened, the name of the file can come in one of two forms:

- "**file.txt**" assumes the file name is file.txt, and it is located in the current program directory.
- "**c:\bill\file.txt**" is the fully qualified file name and includes the directory information.

File Modes

Mode	How Opened	File Exists	File Does Not Exist
'r'	Read-only	Opens that file	Error
'w'	Write-only	Clears the file contents	Creates and opens a new file
'a'	Write-only	File contents left intact and new data appended at file's end	Creates and opens a new file
'r+'	Read and Write	Reads and overwrites from the file's beginning	Error
'w+'	Read and Write	Clears the file contents	Creates and opens a new file
'a+'	Read and Write	File contents left intact and read and write at file's end	Creates and opens a new file

- Text files
 - Default: UTF-8 (a variable-length encoding for Unicode)
- Binary files
 - Different files, depending on language/OS, will have different encodings!
 - Specify the encoding explicitly

```
open("table.csv", "r", encoding="windows-1252")
```

- More about encodings
 - <http://getpython3.com/diveintopython3/strings.html#boring-stuff>

Strings or Bytes?

- If you interact with **text files**, remember:
 - files store things as characters (character encoding)
 - Default encoding: UTF-8
 - All access to text files is via **strings**
 - Read strings from a text file
 - Write strings to a text file
- How about binary files?
 - All access to binary files is via bytes (encoding)
 - More details
 - <http://getpython3.com/diveintopython3/strings.html#boring-stuff>

Reading a File

```
>>> aFile = open("temp.txt", "r")
```

```
>>> for line_str in aFile:  
    print(line_str, end='')
```

First Line
Second Line
Third Line

temp.txt

First Line
Second Line
Third Line

Other Methods to Read Files

- `fileObject.readline()`
 - return the next line as a string.
- `fileObject.readlines()`
 - Return a list of all the lines from the file.
- `fileObject.read(N)`
 - Read **N characters** and returned a single string
 - If N is omitted, the entire file is read and returned as a single string

The `readline()` method

```
>>> aFile = open("temp.txt", "r")
```

```
>>> first_str = aFile.readline()
```

```
>>> first_str  
'First Line\n'
```

```
>>> second_str = aFile.readline()
```

```
>>> second_str  
'Second Line\n'
```

temp.txt

```
First Line  
Second Line  
Third Line
```

The `readlines()` method

```
>>> aFile = open("temp.txt", "r")  
  
>>> file_contents = aFile.readlines()  
  
>>> file_contents  
  
['First Line\n', 'Second Line']
```

temp.txt

```
First Line  
Second Line
```

The read() method

```
>>> aFile = open("temp.txt", "r")
```

```
>>> aFile.read(1)
'F'
```

```
>>> aFile.read(2)
'ir'
```

```
>>> aFile.read()
'st Line\nSecond Line'
```

```
>>> aFile.read(1)
''
```

temp.txt

```
First Line
Second Line
```


Writing a Text File

```
>>> aFile = open("temp.txt", "w")
>>> print("first line", file=aFile)
>>> print("second line_", file=aFile, end='')
>>> print("third line", file=aFile)
>>> aFile.close()
```

What if you don't specify this?

temp.txt

```
first line
second line_third line
```

temp.txt



Close the door behind you!

- Closing the file is important
 - the information in the `fileObject` buffer is “flushed” out of the buffer and into the file on disk

```
fileObject.close()
```

- Python 3:

```
with open("fileToRead.txt") as myFile:  
    for line in myFile:  
        print(line)
```

- File is **automatically opened**
 - Default file mode is **read** & file type is **text**.
- File is **automatically closed** at the end of the for loop.

Other methods to write Text Files

- `fileObject.write(str)`
 - write the string `str` to the file
- `fileObject.writelines(list)`
 - write a list of strings to the file

The `write()` method

```
>>> aFile = open("temp.txt", "w")
>>> aFile.write("First Line\n")
>>> aFile.write("Second Line\n")
>>> aFile.close()
```

temp.txt

```
First Line
Second Line
```

The `writelines()` method

```
>>> aFile = open("temp.txt", "w")
>>> line_list = ["First Line\n", "Second Line\n"]
>>> aFile.writelines(line_list)
>>> aFile.close()
```

temp.txt

```
First Line
Second Line
```

Errors?

- What if the file does not exist?
- Your program should **behave gracefully** if the file cannot be opened.
 - Exception handling
 - `try/except` block

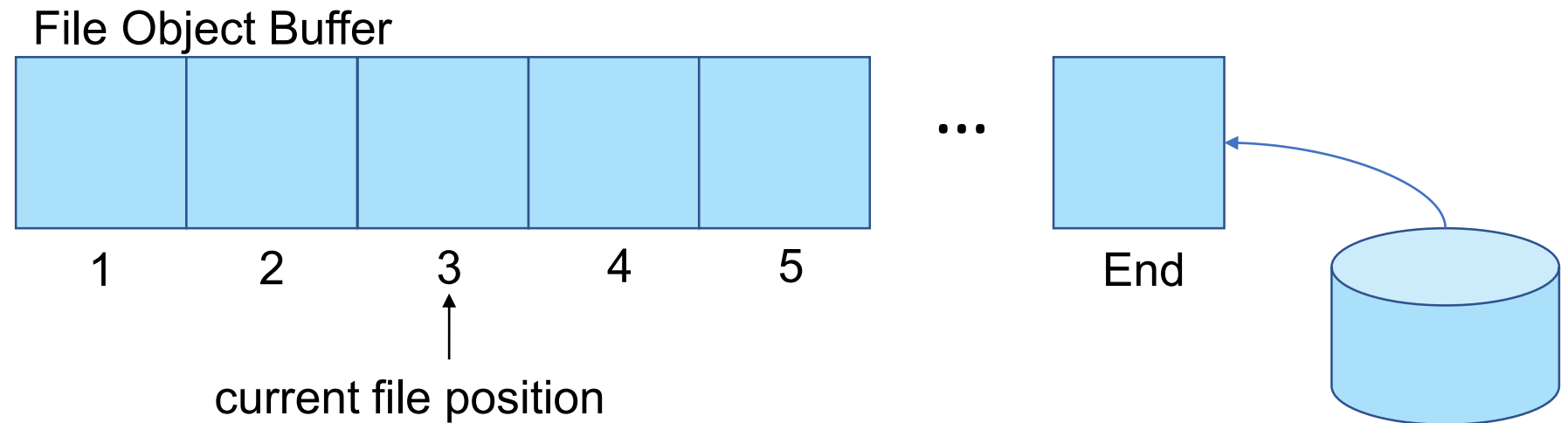
File Facts

Current File Position

- Every file maintains a **current file position**.
- It is the **current position** in the file and indicates what will be read next.
- It is set by the file mode.

Mode	How Opened	File Exists	File Does Not Exist
'r'	Read-only	Opens that file	Error
'w'	Write-only	Clears the file contents	Creates and opens a new file
'a'	Write-only	File contents left intact and new data appended at file's end	Creates and opens a new file
'r+'	Read and Write	Reads and overwrites from the file's beginning	Error
'w+'	Read and Write	Clears the file contents	Creates and opens a new file
'a+'	Read and Write	File contents left intact and read and write at file's end	Creates and opens a new file

- When the disk file is opened, the contents of the file are **copied** into the **buffer** of the file object.
- Think of the file object as a very big list
- The **current file position** is the **present index** to access the list.



The `tell()` method

- The `tell()` method tells you the current file position.
- The positions are in **bytes** from the beginning of the file:

```
fileObject.tell() => 42
```
- This is not necessarily the same as the number of characters
 - depends on encoding, some characters take multiple bytes.
- Notice that, `read()` operates in **characters**

The tell() method - Example

```
>>> aFile = open("temp.txt", "r")
>>> aFile.tell()
0
>>> aFile.read(16)
'Dive into Python'
>>> aFile.read(1)
','
>>> aFile.tell()
17
>>> aFile.read(1)
'是'
>>> aFile.tell()
20
```

Requires three bytes

temp.txt (UTF-8)

Dive into Python 是一本好書!

The `seek()` method

The `seek()` method updates the current file position to where you like (offset in **bytes** from the beginning of the file):

- `fo.seek(0)` # to the beginning of the file
- `fo.seek(100)` # to 100 bytes from beginning

The seek () method

- Counting offset in bytes is a pain.
- `seek ()` has a **optional second argument**:
 - **0**: count from the beginning
 - **1**: count from the current position in the file
 - **2**: count from the end (usually paired with negative offset)

e.g. `fo.seek(-100, 2)`
100 bytes before end of file

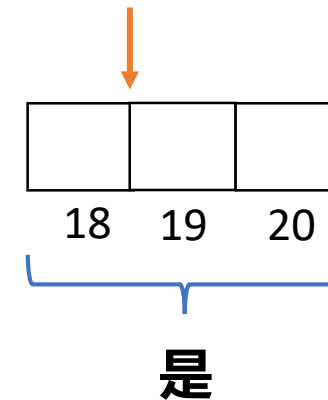
- In **text files**, only seeks relative to the beginning of the file are allowed
 - except `fo.seek(0, 2)`  to the end of the file

The seek () method - Example

```
>>> aFile = open("temp.txt", "r")
>>> aFile.read(17)
'Dive into Python '
>>> aFile.tell()
17
>>> aFile.read(1)
'是'
>>> aFile.tell()
20
>>> aFile.seek(18)
>>> aFile.read(1)
```

temp.txt (UTF-8)

Dive into Python 是一本好書!



UnicodeDecodeError!!!!

- `read()`, `readline()`, `readlines()` move the **current position** forward.
- When you hit the end of the file, every read will just yield "" (empty string)
 - since you are at the end.
- You need to `seek` to the beginning to start again (or close and open, `seek()` is easier).

In this lesson, we have learned:

- The concept of files
- How to interact with files in Python



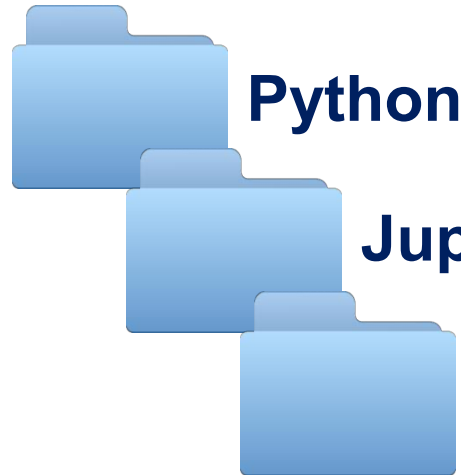
Virtual Environment & Jupyter Notebook



At the end of this lesson, you should be able to:

- Understand the concept of Python Virtual Environment
- Setup Virtual Environment
- Install Jupyter Notebook on Local Machine
- Explore Jupyter Notebook
- Familiarized Markdown Language on Jupyter Notebook

Topic Outline

- 
- Python Virtual Environment**
 - Jupyter Notebook Overview**
 - Installing and Using Jupyter Notebook**

Why do we need Python Virtual Environment

- Python applications will often use various packages and modules that don't come as part of the standard library.
- Applications will sometimes need a specific version of a library, because the application may require that a particular bug has been fixed or the application may be written using an obsolete version of the library's interface.
- Therefore, it may not be possible for one Python installation to meet the requirements of every single application.

Murat Durmus

A Hands-On Introduction to **ESSENTIAL PYTHON** **LIBRARIES AND FRAMEWORKS** (WITH CODE SAMPLES)

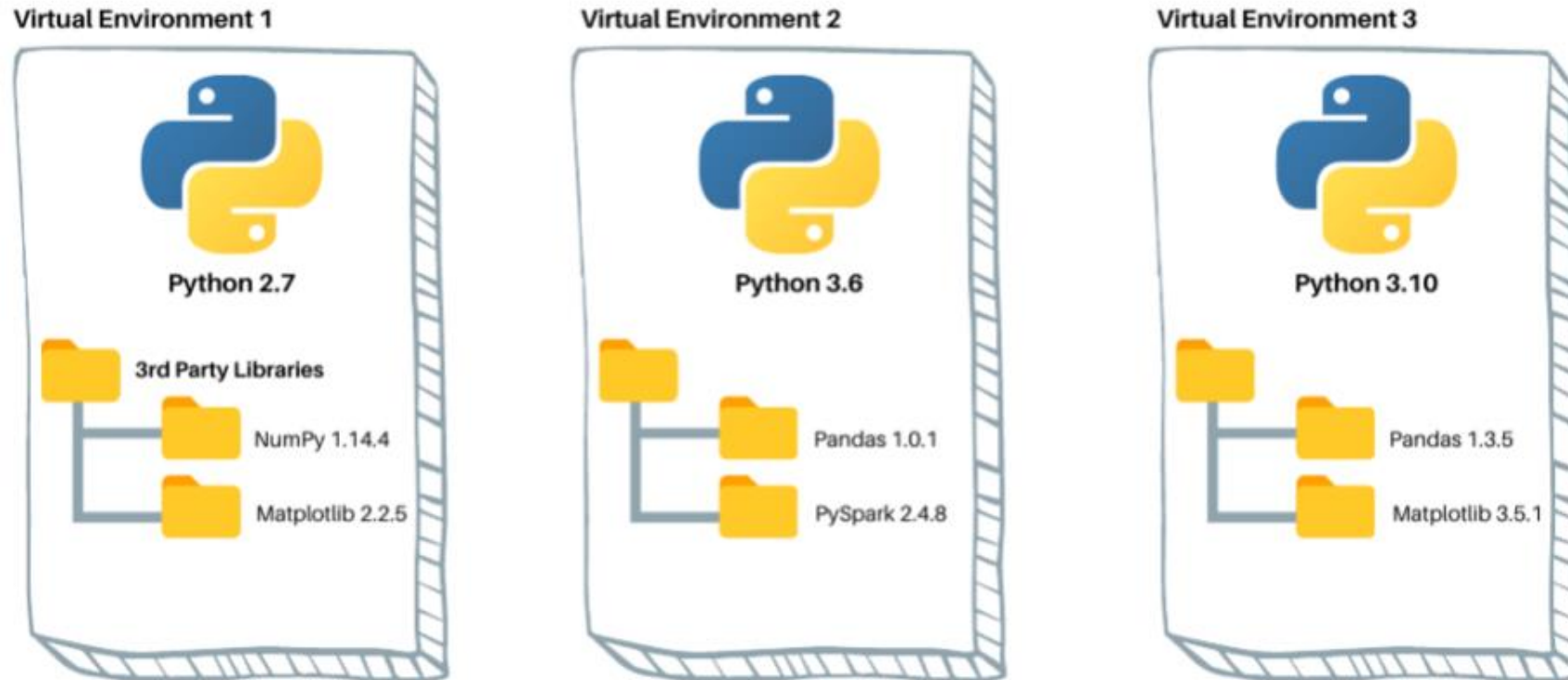


What is Python Virtual Environment

- The solution for this problem is to create a virtual environment for each project.
- Virtual environment is a self-contained directory tree that contains a Python installation for a particular version of Python, plus a number of additional packages.
- Gives you everything you need to run a lightweight yet isolated Python environment.
- Example projects:
 - Data Science
 - Machine Learning
 - Web Development
 - Game Development
 - Chatbot Development
 - IoT & etc...

```
venv\  
|  
|--- Include\  
|  
|--- Lib\  
|   |  
|   |--- site-packages\  
|   |   |  
|   |   |--- pip\  
|   |   |  
|   |   |--- pip-24.2.dist-info\  
|   |  
|--- Scripts\  
|   |--- Activate.ps1  
|   |--- activate  
|   |--- activate.bat  
|   |--- deactivate.bat  
|   |--- pip.exe  
|   |--- pip3.12.exe  
|   |--- pip3.exe  
|   |--- python.exe  
|   |--- pythonw.exe  
|  
|--- pyvenv.cfg
```


What is Python Virtual Environment



dataquest.io

Source: <https://www.dataquest.io/blog/a-complete-guide-to-python-virtual-environments/>

What is Python Virtual Environment

- Virtual Environment Folder Structure:

Folder	Description
Include\	initially empty folder that Python uses to include C header files for packages you might install that depend on C extensions.
Lib\	contains the site-packages\ folder, which is one of the main reasons for creating your virtual environment. This folder is where you'll install external packages that you want to use within your virtual environment.
Scripts\	contains the executable files of your virtual environment. Most notable are the Python interpreter (python.exe), the pip executable (pip.exe), and the activation script for your virtual environment, which comes in a couple of different flavors to allow you to work with different shells.
pyvenv.cfg	a crucial file for your virtual environment. It contains only a couple of key-value pairs that Python uses to set variables in the sys module that determine which Python interpreter and site-packages directory the current Python session will use.

```

venv\
|
| Include\
|
| Lib\
|   |
|   | site-packages\
|   |   |
|   |   | pip\
|   |   |
|   |   | pip-24.2.dist-info\
|   |
|
| Scripts\
|   |
|   | Activate.ps1
|   | activate
|   | activate.bat
|   | deactivate.bat
|   | pip.exe
|   | pip3.12.exe
|   | pip3.exe
|   | python.exe
|   | pythonw.exe
|
| pyvenv.cfg

```

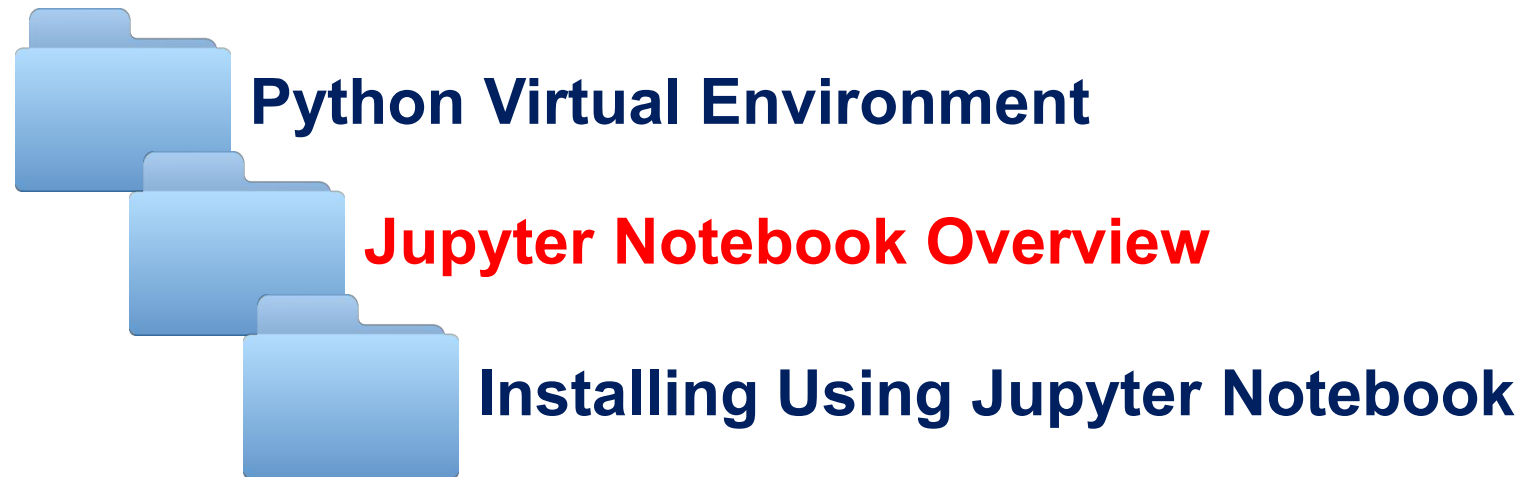
How to setup virtual environment and use it?

The following steps will be shown in the demo:

1. Install Python (pre-requisite)
2. Setup virtual environment
3. Activate virtual environment
4. Install required package
5. Develop a simple application
6. Deactivate the environment

Source: <https://realpython.com/python-virtual-environments-a-primer/#how-can-you-work-with-a-python-virtual-environment>

Topic Outline

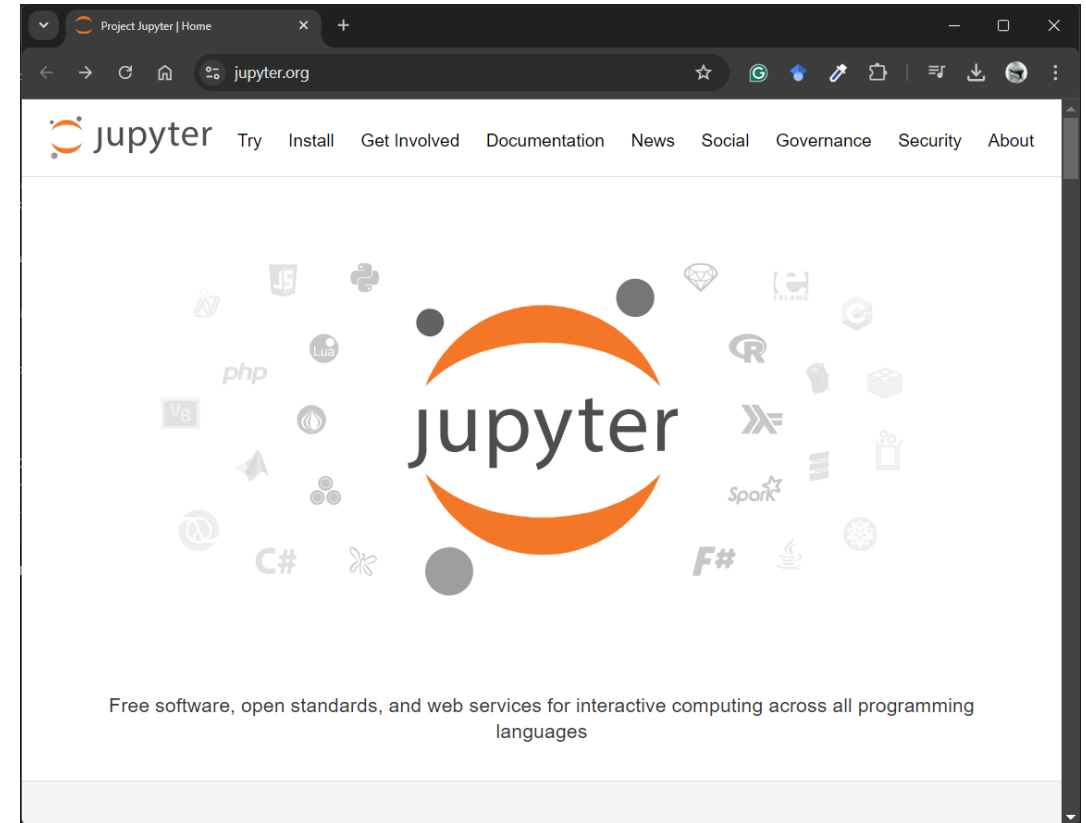


What is Jupyter Notebook

- Free software, open standards, and web services for interactive computing across all programming languages.
- **Notebook** is a document produced by the **Jupyter Notebook App**, which contain both computer **code** (e.g. python) and rich text elements (paragraph, equations, figures, links, etc...), using **Markdown** language.
- Notebook documents are both human-readable documents containing the analysis description and the results (figures, tables, etc..) as well as executable code which can be run to perform some data analysis or simulation.

Source: <https://jupyter.org/>

https://jupyter-notebook-beginner-guide.readthedocs.io/en/latest/what_is_jupyter.html

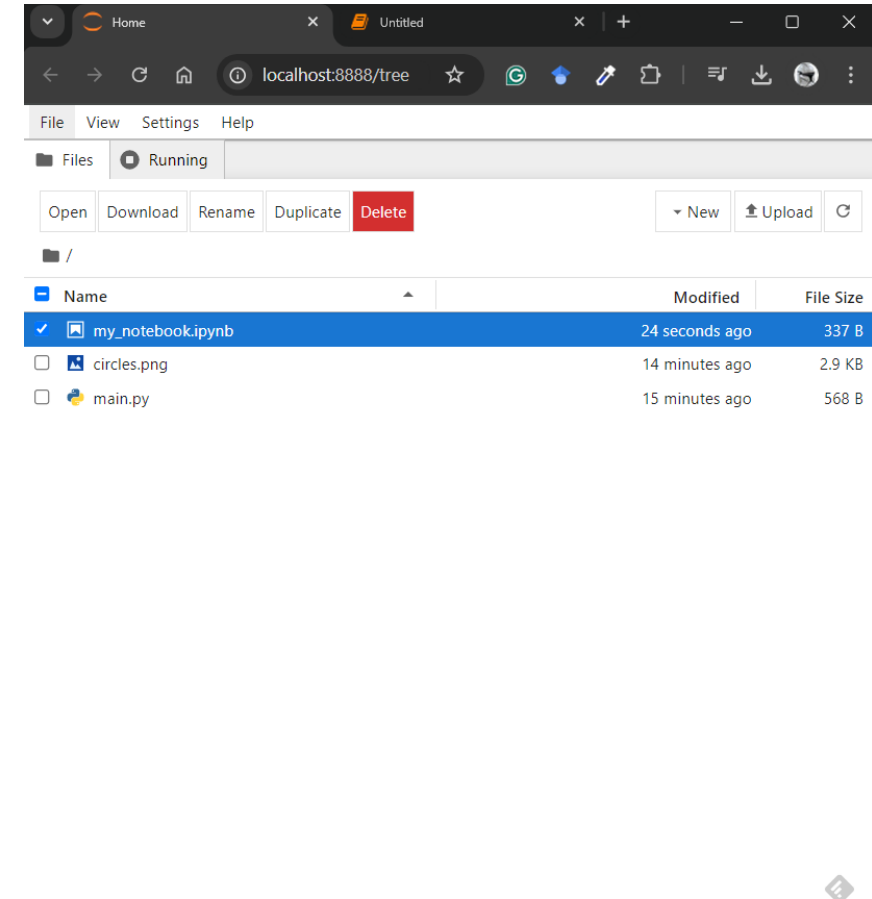


What is Jupyter Notebook App

- **Jupyter Notebook App** is a server-client application that allows editing and running notebook documents via a web browser.
- The Jupyter Notebook App can run on a local machine/host requiring no internet access or can be installed on a remote server and accessed through the internet (e.g. Google Colab).
- A **notebook kernel** is a “computational engine” that executes the code contained in a Notebook document.
- For python, we are using the **ipython kernel** to executes python code. The .ipynb is the file extension for Python notebook.
- Kernels for many other languages exist (official kernels).

Source: <https://jupyter.org/>

https://jupyter-notebook-beginner-guide.readthedocs.io/en/latest/what_is_jupyter.html



What is Markdown

- **Markdown** is a lightweight language with plain text formatting syntax. It is used to create **formatted text** using a plain text editor.
- In Jupyter Notebooks, Markdown is used to add narrative, explanations, and documentation alongside code. This helps in creating well-documented and readable notebooks. ([sample syntax](#))
- To use Markdown in a Jupyter Notebook, you need to create a new cell and set its type to "Markdown" by selecting "Markdown" from the dropdown menu in the toolbar or by pressing M while the cell is selected. ([shortcut](#))

More about Jupyter Notebook:

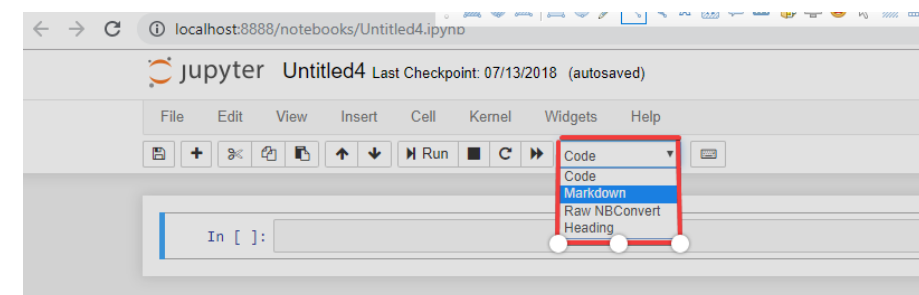
<https://medium.com/edureka/jupyter-notebook-cheat-sheet-88f60d1aca7>

Jupyter Notebook Markdown Cheatsheet

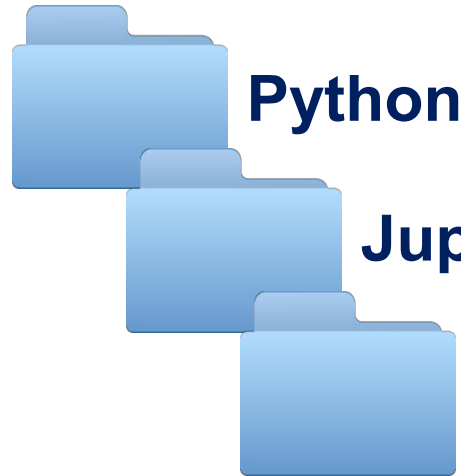
#Header 1 Header 1 -----	Header 1
##Header 2 Header 2 -----	Header 2
###Header 3 Header 3 -----	Header 3
####Header 4 Header 4 -----	Header 4
#####Header 5 Header 5 -----	Header 5
italics _italics_	<i>italics</i>
literal asterisks	*literal asterisks*
bold __bold__	bold
--strikethrough--	strikethrough
1. First item 2. Second item A. Subitem	1. First item 2. Second item A. Subitem
*Item 1 uIndent ~Item 2 u~Item 3	• Item 1 • Item 2 • Item 3
- [x] Done - [] To do	• ☒ Done • ☐ To do
A Line Break ~ ~ ~	A Line Break

From [SqlBak.com](#) with ♥

 [Go to anchor] (#anchor)	Go to anchor									
#uTop Header [Go to header] (#Top-Header)										
https://sqlbak.com										
[Link] (https://sqlbak.com "optional title")	Link									
Click [here] [id] [id]:https://sqlbak.com										
>blockquote text	blockquote text									
python print('hello');	print('hello');									
'inline_code()'										
Left Center Right :--- :--- :--- 1 A C 2 B D	<table><tr><th>Left</th><th>Center</th><th>Right</th></tr><tr><td>1</td><td>A</td><td>C</td></tr><tr><td>2</td><td>B</td><td>D</td></tr></table>	Left	Center	Right	1	A	C	2	B	D
Left	Center	Right								
1	A	C								
2	B	D								
![[alt text]](logo.png "Title") ![[id]] [id]:logo.png "Title"										
\$\$\sqrt{k}\$\$ Inline: \$\sqrt{k}\$	$\sqrt{k}$									
[[img Alt Text]](http://img.youtube.com/vi/ a2CX0w707nc/0.jpg) (https://youtu be/a2CX0w707nc "Video Title")										



Topic Outline

- 
- Python Virtual Environment
 - Jupyter Notebook Overview
 - Installing and Using Jupyter Notebook**

How to install Jupyter Notebook and use it?

The following steps will be shown in the demo:

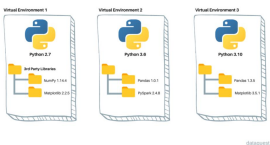
1. Install Jupyter Notebook
2. Start Jupyter Notebook
3. Exploring Jupyter Notebook
 - a. Interactive coding
 - b. Markdown syntax (documentation / reporting)

Source: <https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/What%20is%20the%20Jupyter%20Notebook.html>

In this lesson, we have learned:

- What is Python Virtual Environment and how to create a virtual environment
- What is Jupyter Notebook
- How to install and start a Jupyter notebook
- Markdown language & syntax

References for Images

No.	Slide No.	Image	Reference
1			A Hands-On Introduction to Essential Python Libraries and Frameworks Book Cover [Online Image] https://www.amazon.sg/Introduction-Essential-Libraries-Frameworks-Samples/dp/B0BW2MGYG4
2			Virtual Environment File Structure [Online Image] https://realpython.com/python-virtual-environments-a-primer/#how-can-you-work-with-a-python-virtual-environment
3			Virtual Environments [Online Image] https://www.dataquest.io/blog/a-complete-guide-to-python-virtual-environments/
4			Jupyter Notebook Markdown [Online Image] https://sqlbak.com/blog/jupyter-notebook-markdown-cheatsheet/